

R Notebook

```
library(psych)
```

```
## Warning: package 'psych' was built under R version 4.3.3
```

```
library(mgcv)
```

```
## Warning: package 'mgcv' was built under R version 4.3.3
```

```
## Loading required package: nlme
```

```
## Warning: package 'nlme' was built under R version 4.3.3
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
library(scales)
```

```
##  
## Attaching package: 'scales'
```

```
## The following objects are masked from 'package:psych':  
##  
##   alpha, rescale
```

```
library(latex2exp)  
library(knnflex)  
library(DiagrammeR)
```

```
## Warning: package 'DiagrammeR' was built under R version 4.3.3
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.3
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
## Warning: package 'tidyr' was built under R version 4.3.3
```

```
## Warning: package 'readr' was built under R version 4.3.3
```

```
## — Attaching core tidyverse packages ————— tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.5.1      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.1
## ✓ purrr      1.0.2
```

```
## — Conflicts ————— tidyverse_conflicts() —
## X ggplot2::%+%()      masks psych::%+%()
## X ggplot2::alpha()    masks scales::alpha(), psych::alpha()
## X readr::col_factor() masks scales::col_factor()
## X dplyr::collapse()   masks nlme::collapse()
## X purrr::discard()    masks scales::discard()
## X dplyr::filter()     masks stats::filter()
## X dplyr::lag()        masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to be
come errors
```

```
library(utils)
library(Metrics)
library(dcurves)
```

```
## Warning: package 'dcurves' was built under R version 4.3.3
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##   expand, pack, unpack
##
## Loaded glmnet 4.1-8
```

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.3.3
```

```
##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:dplyr':
##
##   slice
```

```
library(mgcv)
library(caret)
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:Metrics':
##
##   precision, recall
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(ROCR)
library(plyr)
```

```
## -----
## You have loaded plyr after dplyr - this is likely to cause problems.
## If you need functions from both plyr and dplyr, please load plyr first, then dplyr:
## library(plyr); library(dplyr)
## -----
##
## Attaching package: 'plyr'
##
## The following objects are masked from 'package:dplyr':
##
##   arrange, count, desc, failwith, id, mutate, rename, summarise,
##   summarize
##
## The following object is masked from 'package:purrr':
##
##   compact
```

```
library(partykit)
```

```
## Warning: package 'partykit' was built under R version 4.3.3
```

```
## Loading required package: grid
## Loading required package: libcoin
## Loading required package: mvtnorm
```

```
## Warning: package 'mvtnorm' was built under R version 4.3.3
```

```
##
## Attaching package: 'partykit'
##
## The following object is masked from 'package:DiagrammeR':
##
##     get_paths
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.3
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
##
## The following object is masked from 'package:psych':
##
##     outlier
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following object is masked from 'package:Metrics':
##
##     auc
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.3.3
```

```
library(class)
library(car)
```

```
## Warning: package 'car' was built under R version 4.3.3
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##   recode
##
## The following object is masked from 'package:purrr':
##
##   some
##
## The following object is masked from 'package:psych':
##
##   logit
```

```
library(rpart)
```

```
## Warning: package 'rpart' was built under R version 4.3.3
```

```
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.3.3
```

```
library(cutpointr)
```

```
##
## Attaching package: 'cutpointr'
##
## The following objects are masked from 'package:PROC':
##
##   auc, roc
##
## The following objects are masked from 'package:caret':
##
##   precision, recall, sensitivity, specificity
##
## The following objects are masked from 'package:Metrics':
##
##   accuracy, auc, precision, recall
```

```
library(EnvStats)
```

```
## Warning: package 'EnvStats' was built under R version 4.3.3
```

```
## Registered S3 method overwritten by 'EnvStats':
##   method      from
##   print.estimate lava
##
## Attaching package: 'EnvStats'
##
## The following object is masked from 'package:car':
##
##   qqPlot
##
## The following objects are masked from 'package:e1071':
##
##   kurtosis, skewness
##
## The following objects are masked from 'package:stats':
##
##   predict, predict.lm
```

```
# Set the working directory
setwd("C:/Ngoc/MastersThesis/Draft")
```

```
# Load the .csv file
df <- read.csv('diabetes.csv')
head(df)
```

```
##   Pregnancies Glucose BloodPressure SkinThickness Insulin   BMI
## 1           6    148           72           35         0 33.6
## 2           1     85           66           29         0 26.6
## 3           8    183           64            0         0 23.3
## 4           1     89           66           23        94 28.1
## 5           0    137           40           35       168 43.1
## 6           5    116           74            0         0 25.6
##   DiabetesPedigreeFunction Age Outcome
## 1              0.627    50         1
## 2              0.351    31         0
## 3              0.672    32         1
## 4              0.167    21         0
## 5              2.288    33         1
## 6              0.201    30         0
```

Preprocessing data:

```
str(df)
```

```
## 'data.frame':    768 obs. of  9 variables:
## $ Pregnancies      : int  6 1 8 1 0 5 3 10 2 8 ...
## $ Glucose          : int  148 85 183 89 137 116 78 115 197 125 ...
## $ BloodPressure    : int  72 66 64 66 40 74 50 0 70 96 ...
## $ SkinThickness    : int  35 29 0 23 35 0 32 0 45 0 ...
## $ Insulin          : int  0 0 0 94 168 0 88 0 543 0 ...
## $ BMI              : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ DiabetesPedigreeFunction: num  0.627 0.351 0.672 0.167 2.288 ...
## $ Age              : int  50 31 32 21 33 30 26 29 53 54 ...
## $ Outcome          : int  1 0 1 0 1 0 1 0 1 1 ...
```

```
# => The dataset has 768 observations and 9 variables. All variables are in numerical format
```

```
# Since Outcome is supposed to be a categorical variable, we need to convert it into a factor
df$Outcome <- as.factor(df$Outcome)

str(df)
```

```
## 'data.frame':    768 obs. of  9 variables:
## $ Pregnancies      : int  6 1 8 1 0 5 3 10 2 8 ...
## $ Glucose          : int  148 85 183 89 137 116 78 115 197 125 ...
## $ BloodPressure    : int  72 66 64 66 40 74 50 0 70 96 ...
## $ SkinThickness    : int  35 29 0 23 35 0 32 0 45 0 ...
## $ Insulin          : int  0 0 0 94 168 0 88 0 543 0 ...
## $ BMI              : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ DiabetesPedigreeFunction: num  0.627 0.351 0.672 0.167 2.288 ...
## $ Age              : int  50 31 32 21 33 30 26 29 53 54 ...
## $ Outcome          : Factor w/ 2 levels "0","1": 2 1 2 1 2 1 2 1 2 2 ...
```

```
# Descriptive statistics
summary(df)
```

```
## Pregnancies      Glucose      BloodPressure      SkinThickness
## Min.   : 0.000    Min.   : 0.0    Min.   : 0.00    Min.   : 0.00
## 1st Qu.: 1.000    1st Qu.: 99.0    1st Qu.: 62.00    1st Qu.: 0.00
## Median : 3.000    Median :117.0    Median : 72.00    Median :23.00
## Mean   : 3.845    Mean   :120.9    Mean   : 69.11    Mean   :20.54
## 3rd Qu.: 6.000    3rd Qu.:140.2    3rd Qu.: 80.00    3rd Qu.:32.00
## Max.   :17.000    Max.   :199.0    Max.   :122.00    Max.   :99.00
## Insulin      BMI      DiabetesPedigreeFunction      Age
## Min.   : 0.0    Min.   : 0.00    Min.   :0.0780    Min.   :21.00
## 1st Qu.: 0.0    1st Qu.:27.30    1st Qu.:0.2437    1st Qu.:24.00
## Median : 30.5    Median :32.00    Median :0.3725    Median :29.00
## Mean   : 79.8    Mean   :31.99    Mean   :0.4719    Mean   :33.24
## 3rd Qu.:127.2    3rd Qu.:36.60    3rd Qu.:0.6262    3rd Qu.:41.00
## Max.   :846.0    Max.   :67.10    Max.   :2.4200    Max.   :81.00
## Outcome
## 0:500
## 1:268
##
##
##
##
```

```
# Split train : test as 70 : 30
set.seed(6)
train_indices <- createDataPartition(df$Outcome, p = 0.7, list = FALSE)
test_indices <- setdiff(1:nrow(df), train_indices)
# If replace = True, the same data point can appear in both the training and test set

train <- df[train_indices, ]
test  <- df[test_indices, ]

X_train <- df[train_indices, 1:8]
y_train <- df[train_indices, 9]
X_test  <- df[test_indices, 1:8]
y_test  <- df[test_indices, 9]

# Examine the size of the training and test sets
cat("Training set's size:", nrow(train), "\n")
```

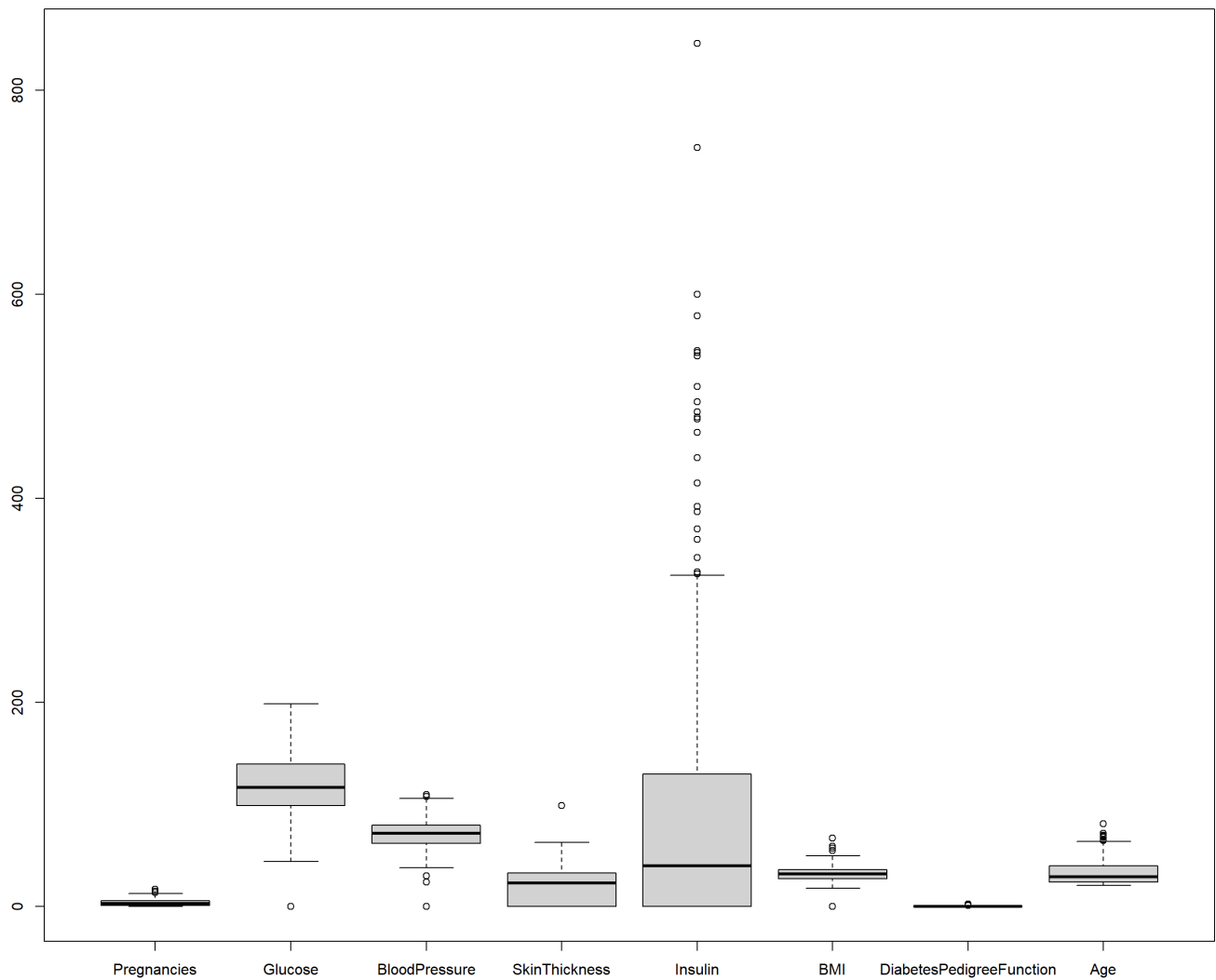
```
## Training set's size: 538
```

```
cat("Test set's size:", nrow(test), "\n")
```

```
## Test set's size: 230
```

```
# # When checking outliers through boxplot / Rosner test / Mahalanobis distance, you only need to check outliers for the numerical feature variables, not the outcome variable

# Check for outliers in numerical feature variables of the training set
boxplot(train[, 1:8], na.action = NULL) # na.action = NULL means to ignore NA values in the dataset
```

```
# => From the boxplot, it seems like Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age may have outliers
```

```
# Use Rosner test to re - check for outliers on these variables
```

```
rosnerTest(train$Pregnancies, k = 2) # k is the maximum number of suspected outlier
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$Pregnancies
##
## Sample Size:                538
##
## Test Statistics:             R.1 = 3.914316
##                             R.2 = 3.369772
##
## Test Statistic Parameter:    k = 2
##
## Alternative Hypothesis:      Up to 2 observations are not
##                             from the same Distribution.
##
## Type I Error:                5%
##
## Number of Outliers Detected: 1
##
##   i   Mean.i    SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1 0 3.892193 3.348684   17   114 3.914316   3.882435   TRUE
## 2 1 3.867784 3.303552   15    64 3.369772   3.881947  FALSE
```

```
rosnerTest(train$Glucose, k = 4)
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$Glucose
##
## Sample Size:                538
##
## Test Statistics:            R.1 = 3.820465
##                             R.2 = 3.877169
##                             R.3 = 3.936474
##                             R.4 = 2.552531
##
## Test Statistic Parameter:   k = 4
##
## Alternative Hypothesis:     Up to 4 observations are not
##                             from the same Distribution.
##
## Type I Error:               5%
##
## Number of Outliers Detected: 3
##
##   i   Mean.i   SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1 0 120.9888 31.66862    0     53 3.820465   3.882435   TRUE
## 2 1 121.2142 31.26358    0    131 3.877169   3.881947   TRUE
## 3 2 121.4403 30.85002    0    346 3.936474   3.881458   TRUE
## 4 3 121.6673 30.42756   44     44 2.552531   3.880967  FALSE
```

```
rosnerTest(train$BloodPressure, k = 11)
```

```
## Warning in rosnerTest(train$BloodPressure, k = 11): The true Type I error may be larger than assumed.
## Although the help file for 'rosnerTest' has a table with information
## on the estimated Type I error level,
## simulations were not run for k > 10 or k > floor(n/2).
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$BloodPressure
##
## Sample Size:                538
##
## Test Statistics:            R.1  = 3.519101
##                             R.2  = 3.563781
##                             R.3  = 3.610207
##                             R.4  = 3.658496
##                             R.5  = 3.708776
##                             R.6  = 3.761189
##                             R.7  = 3.815889
##                             R.8  = 3.873047
##                             R.9  = 3.932853
##                             R.10 = 3.995519
##                             R.11 = 4.061280
##
## Test Statistic Parameter:   k = 11
##
## Alternative Hypothesis:     Up to 11 observations are not
##                             from the same Distribution.
##
## Type I Error:              5%
##
## Number of Outliers Detected: 11
##
##      i   Mean.i    SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1    0 68.57063 19.48527    0      5 3.519101   3.882435   TRUE
## 2    1 68.69832 19.27681    0     13 3.563781   3.881947   TRUE
## 3    2 68.82649 19.06442    0     37 3.610207   3.881458   TRUE
## 4    3 68.95514 18.84795    0     56 3.658496   3.880967   TRUE
## 5    4 69.08427 18.62724    0    123 3.708776   3.880476   TRUE
## 6    5 69.21388 18.40213    0    185 3.761189   3.879983   TRUE
## 7    6 69.34398 18.17243    0    187 3.815889   3.879490   TRUE
## 8    7 69.47458 17.93796    0    190 3.873047   3.878995   TRUE
## 9    8 69.60566 17.69851    0    211 3.932853   3.878500   TRUE
## 10   9 69.73724 17.45386    0    234 3.995519   3.878003   TRUE
## 11  10 69.86932 17.20377    0    243 4.061280   3.877505   TRUE
```

```
rosnerTest(train$SkinThickness, k = 2)
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$SkinThickness
##
## Sample Size:                538
##
## Test Statistics:            R.1 = 4.823004
##                             R.2 = 2.666285
##
## Test Statistic Parameter:   k = 2
##
## Alternative Hypothesis:     Up to 2 observations are not
##                             from the same Distribution.
##
## Type I Error:               5%
##
## Number of Outliers Detected: 1
##
##   i   Mean.i    SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1 0 20.85130 16.20332   99   396 4.823004   3.882435   TRUE
## 2 1 20.70577 15.86260   63   308 2.666285   3.881947  FALSE
```

```
rosnerTest(train$Insulin, k = 11)
```

```
## Warning in rosnerTest(train$Insulin, k = 11): The true Type I error may be larger than assumed.
## Although the help file for 'rosnerTest' has a table with information
## on the estimated Type I error level,
## simulations were not run for k > 10 or k > floor(n/2).
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$Insulin
##
## Sample Size:                538
##
## Test Statistics:            R.1  = 6.414562
##                             R.2  = 5.790937
##                             R.3  = 4.688685
##                             R.4  = 4.599568
##                             R.5  = 4.379181
##                             R.6  = 4.445398
##                             R.7  = 4.505598
##                             R.8  = 4.301793
##                             R.9  = 4.232180
##                             R.10 = 4.309904
##                             R.11 = 4.287991
##
## Test Statistic Parameter:   k = 11
##
## Alternative Hypothesis:     Up to 11 observations are not
##                             from the same Distribution.
##
## Type I Error:               5%
##
## Number of Outliers Detected: 11
##
##      i  Mean.i      SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1  0 82.62268 119.00693   846     11 6.414562   3.882435   TRUE
## 2  1 81.20112 114.45452   744    166 5.790937   3.881947   TRUE
## 3  2 79.96455 110.91287   600    401 4.688685   3.881458   TRUE
## 4  3 78.99252 108.70749   579    281 4.599568   3.880967   TRUE
## 5  4 78.05618 106.62811   545    200 4.379181   3.880476   TRUE
## 6  5 77.18011 104.78699   543      6 4.445398   3.879983   TRUE
## 7  6 76.30451 102.91542   540    454 4.505598   3.879490   TRUE
## 8  7 75.43126 101.02038   510    529 4.301793   3.878995   TRUE
## 9  8 74.61132  99.33147   495     79 4.232180   3.878500   TRUE
## 10 9 73.81664  97.72454   495    135 4.309904   3.878003   TRUE
## 11 10 73.01894  96.07786   485    109 4.287991   3.877505   TRUE
```

```
rosnerTest(train$BMI, k = 9)
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$BMI
##
## Sample Size:                538
##
## Test Statistics:            R.1 = 4.609601
##                             R.2 = 4.263247
##                             R.3 = 4.341599
##                             R.4 = 4.424437
##                             R.5 = 4.512205
##                             R.6 = 4.605414
##                             R.7 = 4.704648
##                             R.8 = 4.050801
##                             R.9 = 3.801044
##
## Test Statistic Parameter:   k = 9
##
## Alternative Hypothesis:     Up to 9 observations are not
##                             from the same Distribution.
##
## Type I Error:               5%
##
## Number of Outliers Detected: 8
##
##   i   Mean.i   SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1 0 31.95167 7.625025 67.1    127 4.609601  3.882435   TRUE
## 2 1 31.88622 7.479327  0.0     7 4.263247  3.881947   TRUE
## 3 2 31.94571 7.358051  0.0    37 4.341599  3.881458   TRUE
## 4 3 32.00542 7.233784  0.0   102 4.424437  3.880967   TRUE
## 5 4 32.06536 7.106360  0.0   257 4.512205  3.880476   TRUE
## 6 5 32.12552 6.975598  0.0   362 4.605414  3.879983   TRUE
## 7 6 32.18590 6.841298  0.0   490 4.704648  3.879490   TRUE
## 8 7 32.24652 6.703238 59.4   308 4.050801  3.878995   TRUE
## 9 8 32.19528 6.604690 57.3   469 3.801044  3.878500  FALSE
```

```
rosnerTest(train$DiabetesPedigreeFunction, k = 7)
```

```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$DiabetesPedigreeFunction
##
## Sample Size:                538
##
## Test Statistics:            R.1 = 5.770535
##                             R.2 = 5.685133
##                             R.3 = 5.740955
##                             R.4 = 5.440159
##                             R.5 = 4.242733
##                             R.6 = 4.208047
##                             R.7 = 3.515819
##
## Test Statistic Parameter:    k = 7
##
## Alternative Hypothesis:      Up to 7 observations are not
##                               from the same Distribution.
##
## Type I Error:                5%
##
## Number of Outliers Detected: 6
##
##   i   Mean.i   SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1 0 0.4782825 0.3364883 2.420    308 5.770535   3.882435   TRUE
## 2 1 0.4746667 0.3261724 2.329    166 5.685133   3.881947   TRUE
## 3 2 0.4712071 0.3164618 2.288     2 5.740955   3.881458   TRUE
## 4 3 0.4678112 0.3068272 2.137    256 5.440159   3.880967   TRUE
## 5 4 0.4646854 0.2984667 1.731    257 4.242733   3.880476   TRUE
## 6 5 0.4623096 0.2936494 1.698    431 4.208047   3.879983   TRUE
## 7 6 0.4599868 0.2889834 1.476    232 3.515819   3.879490   FALSE
```

```
rosnerTest(train$Age, k = 2)
```



```
##
## Results of Outlier Test
## -----
##
## Test Method:                Rosner's Test for Outliers
##
## Hypothesized Distribution:   Normal
##
## Data:                       train$Age
##
## Sample Size:                538
##
## Test Statistics:             R.1 = 4.028763
##                             R.2 = 3.323853
##
## Test Statistic Parameter:    k = 2
##
## Alternative Hypothesis:      Up to 2 observations are not
##                             from the same Distribution.
##
## Type I Error:                5%
##
## Number of Outliers Detected: 1
##
##   i   Mean.i    SD.i Value Obs.Num   R.i+1 lambda.i+1 Outlier
## 1 0 33.29554 11.84097   81   317 4.028763   3.882435   TRUE
## 2 1 33.20670 11.67118   72   312 3.323853   3.881947   FALSE
```

=> Pregnancies, SkinThickness and Age each has 1 outlier. Glucose has 3 outliers. BMI has 8 outliers. DiabetesPedigreeFunction has 6 outliers. For BloodPressure and Insulin, the test indicates that each of them has at Least 11 outliers. However, the Rosner test doesn't run simulations for $k > 10$, so we can't be certain about the number of outliers that BloodPressure and Insulin have

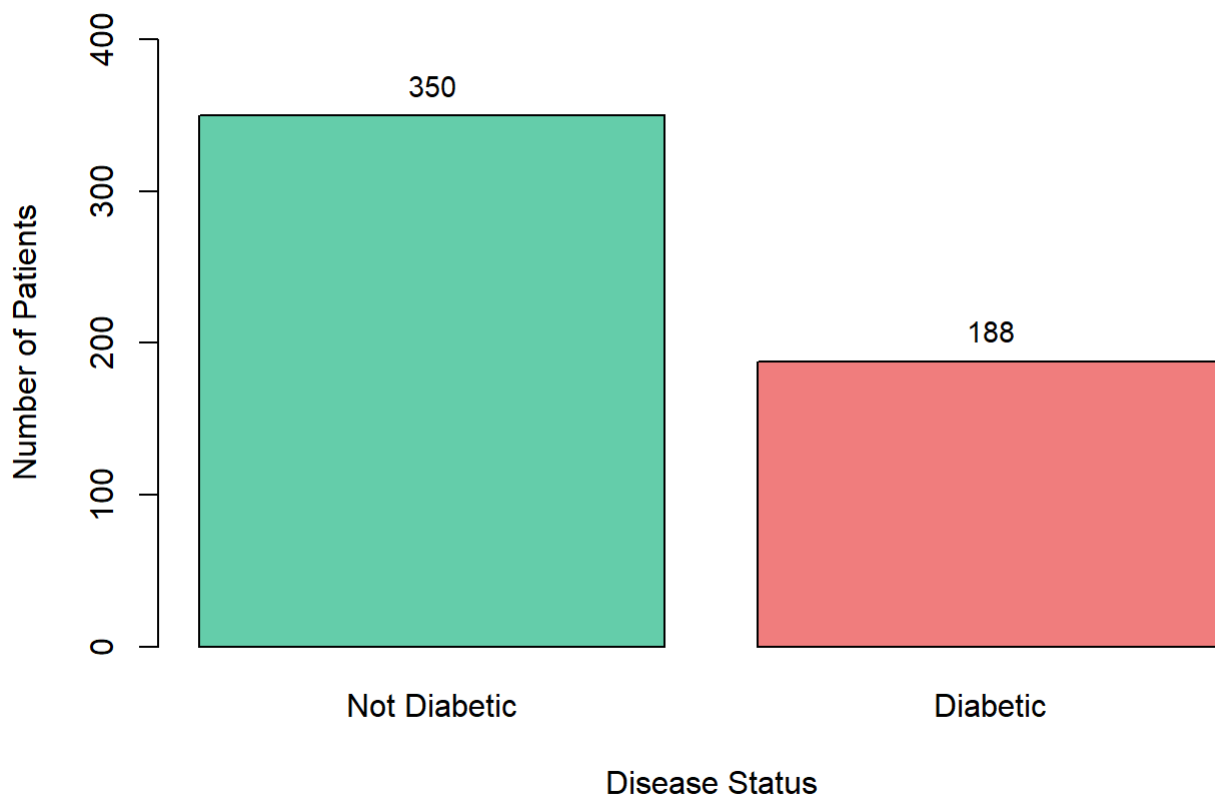
```
# Find columns with NA values
colSums(is.na(train))[colSums(is.na(train))>0]
```

```
## named numeric(0)
```

=> No missing values

```
## Check for class distribution in training set
count <- as.numeric(table(y_train))
custom_labels <- ifelse(names(table(y_train)) == "1", "Diabetic", "Not Diabetic")
bp <- barplot(table(y_train), names.arg = custom_labels, main = "Class distribution in Training Set", col = c("aquamarine3", "lightcoral"), ylim = c(0, 400), xlab = "Disease Status", ylab = "Number of Patients")
text(x = bp, y = count, label = count, cex = 0.9, pos = 3, col = "black")
```

Class distribution in Training Set



-> There is only mild class imbalance (around 65:35 ratio between the two classes)

```
# Only standardize numerical feature variables, not the outcome variable
X_test_unscaled <- X_test

# Standardize the training set
X_train <- scale(X_train, center = TRUE, scale = TRUE)

# Apply the same standardization method on the test set
train_mean <- attr(X_train, "scaled:center") # Same as colMeans(X_train)
train_sd <- attr(X_train, "scaled:scale")    # Same as apply(X_train, 2, sd)

X_test <- scale(X_test, center = train_mean, scale= train_sd)

# Change format from matrix to dataframe
X_train <- as.data.frame(X_train)
train <- cbind(X_train, Outcome = y_train)

X_test <- as.data.frame(X_test)
```

A/ Find training and test set AUC for different classification methods:

1. Logistic regression model:

```
# Fit the logistic regression model
glm_model <- glm(Outcome ~ ., data = train, family = binomial(link = "logit"))
```

```
# Show model's coefficients
summary(glm_model)
```

```
##
## Call:
## glm(formula = Outcome ~ ., family = binomial(link = "logit"),
##      data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -0.87198    0.11713  -7.444 9.74e-14 ***
## Pregnancies      0.42212    0.13392   3.152  0.00162 **
## Glucose         1.24298    0.14807   8.395 < 2e-16 ***
## BloodPressure  -0.32806    0.11812  -2.777  0.00548 **
## SkinThickness   0.01126    0.13699   0.082  0.93451
## Insulin        -0.15929    0.12875  -1.237  0.21603
## BMI             0.66168    0.13993   4.729 2.26e-06 ***
## DiabetesPedigreeFunction 0.35204    0.12329   2.855  0.00430 **
## Age            0.22073    0.13285   1.661  0.09662 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 696.28  on 537  degrees of freedom
## Residual deviance: 495.21  on 529  degrees of freedom
## AIC: 513.21
##
## Number of Fisher Scoring iterations: 5
```

```
y_glm_train_pred = predict.glm(glm_model, X_train, type = 'response') # y_pred = P(Y = 1 | X)
y_glm_test_pred = predict.glm(glm_model, X_test, type = 'response')

# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(response = y_train, predictor = y_glm_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, y_glm_test_pred)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 0.8441337
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.8106786 ; 0.8771204 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.8283333
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7704167 ; 0.8788354 ]
```

```
# Array of false positive rates, true positive rates and thresholds
fpr_glm_test <- 1 - roc_test$specificities
tpr_glm_test <- roc_test$sensitivities
thresholds_glm_test <- roc_test$thresholds
# fpr[i] and tpr[i] are the false positive rate and true positive rate at the threshold value thresholds[i]
```

2. Regularized Logistic regression model:

```
# Fit the logistic regression model with LASSO regularization
glm_lasso_model <- cv.glmnet(x = as.matrix(X_train), y = y_train, alpha = 1, family = "binomial")
# alpha = 1 for LASSO (L1 regularization); family = "binomial" uses the logit link; cv.glmnet uses cross validation to find the optimal lambda

# Show model's coefficients
glm_lasso_coef <- coef(glm_lasso_model, s = "lambda.min") # Obtain the coefficients
glm_lasso_coef
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##                               s1
## (Intercept)                -0.8482818
## Pregnancies                  0.3862777
## Glucose                     1.1683581
## BloodPressure               -0.2708689
## SkinThickness                .
## Insulin                    -0.1009772
## BMI                        0.5957112
## DiabetesPedigreeFunction    0.3048979
## Age                        0.2013032
```

```

y_glm_lasso_train_pred = predict(glm_lasso_model, as.matrix(X_train), s = "lambda.min", type = 'response') # y_pred = P(Y = 1 | X)
y_glm_lasso_test_pred = predict(glm_lasso_model, as.matrix(X_test), s = "lambda.min", type = 'response')
# -> Predictions use the optimal lambda found by cross validation

# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(y_train, as.vector(y_glm_lasso_train_pred))

```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```

auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, as.vector(y_glm_lasso_test_pred))

```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```

auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")

```

```
## AUC on training set: 0.8442249
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.8112264 ; 0.8757036 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.8283333
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7709854 ; 0.8767521 ]
```

```

# Array of false positive rates and true positive rates
fpr_glm_lasso_test <- 1 - roc_test$specificities
tpr_glm_lasso_test <- roc_test$sensitivities

```

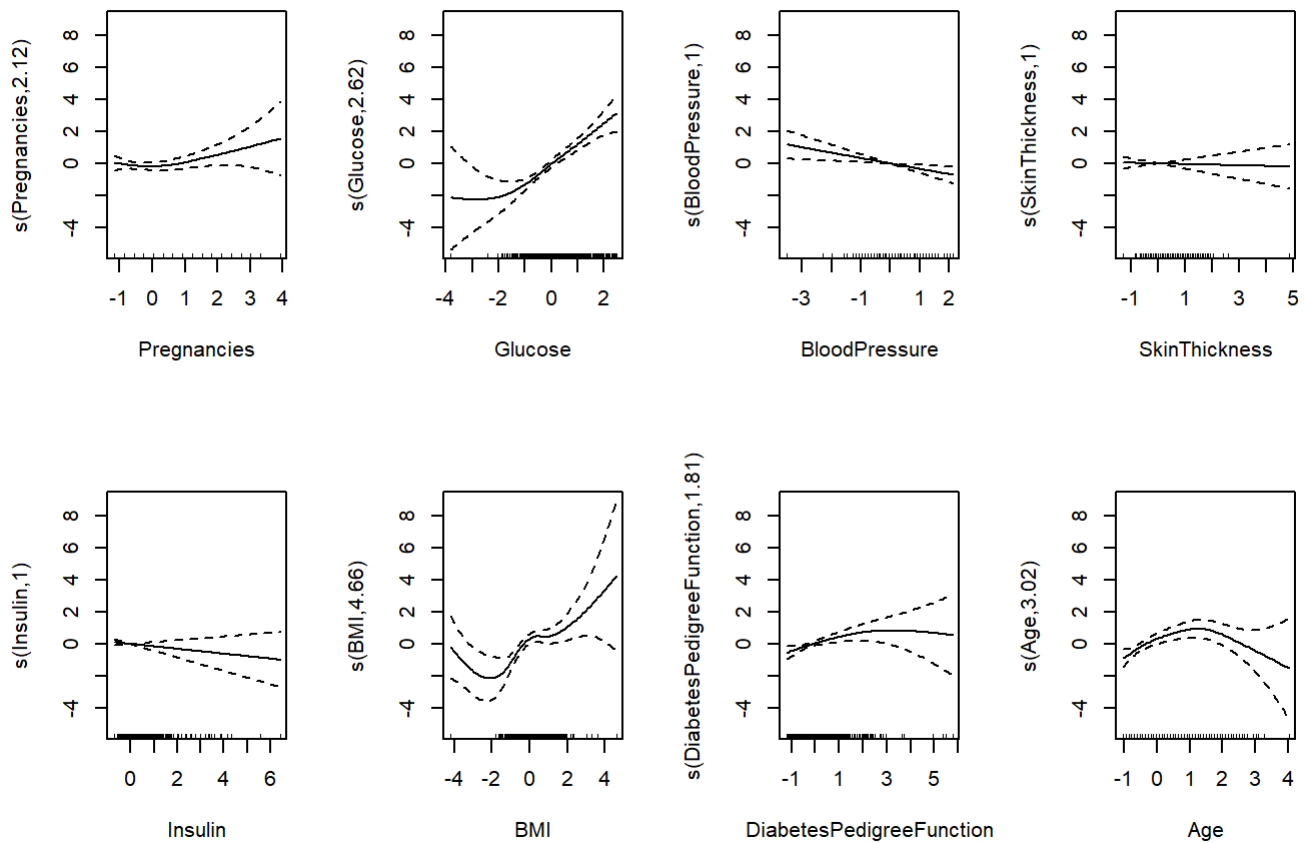
3. Generalized Additive Model:

```
# Fit the GAM model
gam_model <- gam(Outcome ~ s(Pregnancies, k = 15, bs = 'cr') +
  s(Glucose, k = 15, bs = 'cr') +
  s(BloodPressure, k = 15, bs = 'cr') +
  s(SkinThickness, k = 15, bs = 'cr') +
  s(Insulin, k = 15, bs = 'cr') +
  s(BMI, k = 15, bs = 'cr') +
  s(DiabetesPedigreeFunction, k = 15, bs = 'cr') +
  s(Age, k = 15, bs = 'cr'),
  family = binomial(link = "logit"), data = train, method = 'REML', fit = TRUE)

summary(gam_model)
```

```
##
## Family: binomial
## Link function: logit
##
## Formula:
## Outcome ~ s(Pregnancies, k = 15, bs = "cr") + s(Glucose, k = 15,
##   bs = "cr") + s(BloodPressure, k = 15, bs = "cr") + s(SkinThickness,
##   k = 15, bs = "cr") + s(Insulin, k = 15, bs = "cr") + s(BMI,
##   k = 15, bs = "cr") + s(DiabetesPedigreeFunction, k = 15,
##   bs = "cr") + s(Age, k = 15, bs = "cr")
##
## Parametric coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.9507      0.1303  -7.299  2.9e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df Chi.sq  p-value
## s(Pregnancies)    2.125  2.682   3.772 0.191803
## s(Glucose)         2.617  3.377  68.320 < 2e-16 ***
## s(BloodPressure)   1.000  1.000   7.184 0.007358 **
## s(SkinThickness)   1.000  1.000   0.077 0.781387
## s(Insulin)         1.000  1.000   1.285 0.256944
## s(BMI)             4.660  5.648  26.040 0.000187 ***
## s(DiabetesPedigreeFunction) 1.807  2.270   9.230 0.012339 *
## s(Age)            3.020  3.832  15.924 0.003097 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.384   Deviance explained = 35.2%
## -REML = 234.42   Scale est. = 1         n = 538
```

```
# Plot GAM model
par(mfrow = c(2,4))
plot(gam_model)
```



```
y_gam_train_pred = predict.gam(gam_model, X_train, type = 'response') # y_pred = P(Y = 1 / X)
y_gam_test_pred = predict.gam(gam_model, X_test, type = 'response')

# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(y_train, y_gam_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, y_gam_test_pred)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 0.8741945
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.8403248 ; 0.901492 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.8475833
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7925687 ; 0.8952542 ]
```

```
# Array of false positive rates and true positive rates
```

```
fpr_gam_test <- 1 - roc_test$specificities
```

```
tpr_gam_test <- roc_test$sensitivities
```

4. Decision Tree model:

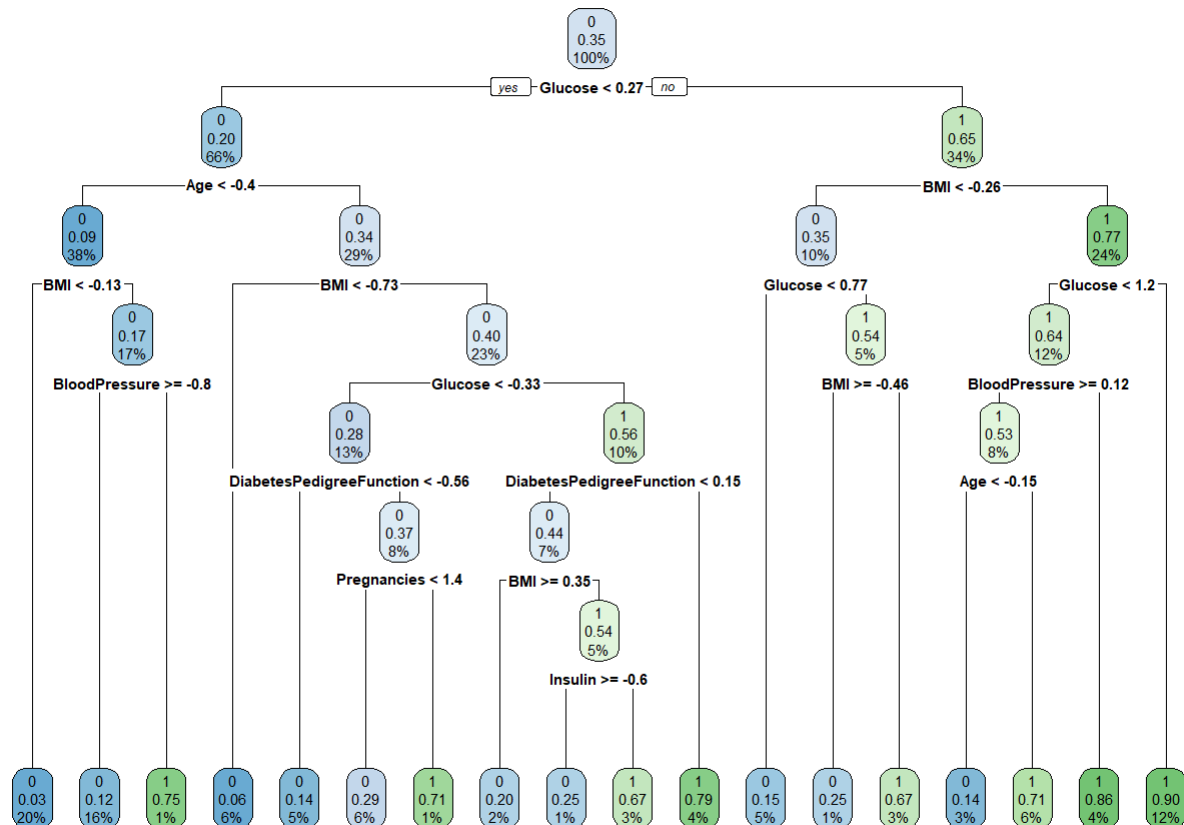
```
# Fit the initial decision tree model
```

```
dtree_model <- rpart(Outcome ~ ., data = train, method = "class", parms = list(split = "gin  
i"), control = rpart.control(cp = 0.006)) # cp is the complexity parameter  
printcp(dtree_model)
```

```
##  
## Classification tree:  
## rpart(formula = Outcome ~ ., data = train, method = "class",  
##      parms = list(split = "gini"), control = rpart.control(cp = 0.006))  
##  
## Variables actually used in tree construction:  
## [1] Age                      BloodPressure          BMI  
## [4] DiabetesPedigreeFunction Glucose              Insulin  
## [7] Pregnancies  
##  
## Root node error: 188/538 = 0.34944  
##  
## n= 538  
##  
##      CP nsplit rel error  xerror    xstd  
## 1 0.2819149      0  1.00000 1.00000 0.058825  
## 2 0.0851064      1  0.71809 0.79787 0.055324  
## 3 0.0177305      2  0.63298 0.75000 0.054257  
## 4 0.0159574      5  0.57979 0.76596 0.054624  
## 5 0.0124113      7  0.54787 0.74468 0.054132  
## 6 0.0106383     13  0.45745 0.70213 0.053089  
## 7 0.0079787     15  0.43617 0.70745 0.053224  
## 8 0.0060000     17  0.42021 0.71277 0.053357
```



```
# Plot decision tree
rpart.plot(dtrees_model)
```



```
y_dtree_train_pred = predict(dtrees_model, X_train, type = 'prob')[,2] # y_pred = P(Y = 1 / X)
```

```
y_dtree_test_pred = predict(dtrees_model, X_test, type = 'prob')[,2]
```

```
# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(y_train, y_dtree_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)
```

```
# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, y_dtree_test_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 0.8931763
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.8619747 ; 0.9212253 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.81325
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7534573 ; 0.8678406 ]
```

```
# Array of false positive rates and true positive rates
fpr_dtree_test <- 1 - roc_test$specificities
tpr_dtree_test <- roc_test$sensitivities
```

5. Random Forest model:

```
# Fit the Random Forest model
rf_model <- randomForest(X_train, as.factor(y_train), importance=TRUE, ntree = 500, replace =
TRUE)
print(rf_model)
```

```
##
## Call:
## randomForest(x = X_train, y = as.factor(y_train), ntree = 500,      replace = TRUE, impor
tance = TRUE)
##              Type of random forest: classification
##              Number of trees: 500
## No. of variables tried at each split: 2
##
##              OOB estimate of  error rate: 23.05%
## Confusion matrix:
##      0   1 class.error
## 0 301  49  0.1400000
## 1   75 113  0.3989362
```

```
y_rf_train_pred = predict(rf_model, X_train, type = 'prob')[,2] #  $y_{pred} = P(Y = 1 \mid X)$ 
y_rf_test_pred = predict(rf_model, X_test, type = 'prob')[,2]

# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(y_train, y_rf_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)
```

```
## Warning in ci.auc.roc(roc_train, method = "bootstrap", conf.level = 0.95, :
## ci.auc() of a ROC curve with AUC == 1 is always 1-1 and can be misleading.
```

```
# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, y_rf_test_pred)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 1
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 1 ; 1 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.831375
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.774625 ; 0.8809208 ]
```

```
# Array of false positive rates and true positive rates
fpr_rf_test <- 1 - roc_test$specificities
tpr_rf_test <- roc_test$sensitivities
```

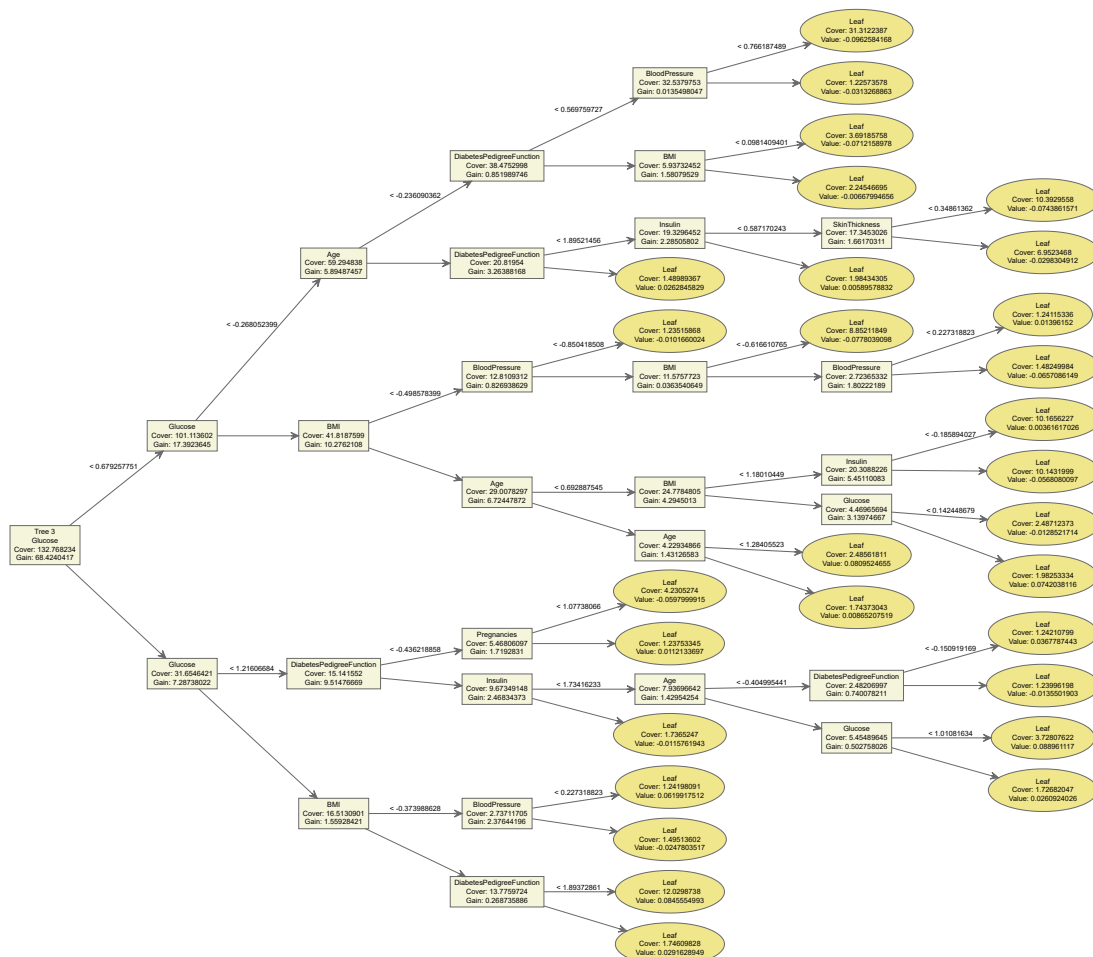
6. Extreme Gradient Boosting model:

Fit the XGBoost model

```
xgb_model <- xgboost(data = as.matrix(X_train), label = as.numeric(as.character(y_train)), nr
ounds = 100, eta = 0.06, objective = "binary:logistic", verbose = 0) # gamma is the regulariz
ation parameter for tree based models
```

Plot the third individual decision tree in the model

```
xgb.plot.tree(model = xgb_model, trees = 3, render = TRUE)
```



```
y_xgb_train_pred = predict(xgb_model, as.matrix(X_train), type = 'prob') # y_pred = P(Y = 1
/ X)
```

```
y_xgb_test_pred = predict(xgb_model, as.matrix(X_test), type = 'prob')
```

Compute the AUC and 95% confidence interval of the AUC for the training set

```
roc_train <- pROC::roc(y_train, y_xgb_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
```

```
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)
```

Compute the AUC and 95% confidence interval of the AUC for the test set

```
roc_test <- pROC::roc(y_test, y_xgb_test_pred)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 0.9998632
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.9995289 ; 1 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.8219167
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7664875 ; 0.8752583 ]
```

```
# Array of false positive rates and true positive rates
fpr_xgb_test <- 1 - roc_test$specificities
tpr_xgb_test <- roc_test$sensitivities
```

7. Support Vector Machine model:

```
# Fit the SVM model
svm_model <- svm(Outcome ~ ., data = train, type = "C-classification", kernel = "linear", probability = TRUE, cost = 0.6)
```

```
y_svm_train_pred = attr(predict(svm_model, X_train, probability = TRUE), "probabilities")[,1]
# y_pred = P(Y = 1 | X)
y_svm_test_pred = attr(predict(svm_model, X_test, probability = TRUE), "probabilities")[,1]

# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(y_train, y_svm_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, y_svm_test_pred)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 0.8427204
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.8072755 ; 0.8764236 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.8270833
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7726583 ; 0.8771687 ]
```

```
# Array of false positive rates and true positive rates
fpr_svm_test <- 1 - roc_test$specificities
tpr_svm_test <- roc_test$sensitivities
```

8. K - Nearest Neighbor model:

```
# Fit the KNN classifier model
knn_model <- knn3(Outcome ~ ., data = train, k = 45)
```

```
y_knn_train_pred = predict(knn_model, X_train, type = 'prob')[,2] # y_pred = P(Y = 1 | X) #
y_pred = P(Y = 1 | X)
y_knn_test_pred = predict(knn_model, X_test, type = 'prob')[,2]

# Compute the AUC and 95% confidence interval of the AUC for the training set
roc_train <- pROC::roc(y_train, y_knn_train_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_train <- pROC::auc(roc_train)
ci_train <- ci.auc(roc_train, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

# Compute the AUC and 95% confidence interval of the AUC for the test set
roc_test <- pROC::roc(y_test, y_knn_test_pred)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
auc_test <- pROC::auc(roc_test)
ci_test <- ci.auc(roc_test, method = "bootstrap", conf.level = 0.95, boot.n = 2000)

cat("AUC on training set: ", auc_train, "\n")
```

```
## AUC on training set: 0.8374316
```

```
cat("95% CI of AUC on training set: [", ci_train[1], " ; ", ci_train[3], "] \n")
```

```
## 95% CI of AUC on training set: [ 0.8029761 ; 0.8719759 ]
```

```
cat("AUC on test set: ", auc_test, "\n")
```

```
## AUC on test set: 0.8145
```

```
cat("95% CI of AUC on test set: [", ci_test[1], " ; ", ci_test[3], "] \n")
```

```
## 95% CI of AUC on test set: [ 0.7586594 ; 0.869376 ]
```

```
# Array of false positive rates and true positive rates
fpr_knn_test <- 1 - roc_test$specificities
tpr_knn_test <- roc_test$sensitivities
```

B/ Choosing decision threshold by ROC curve and decision curve:

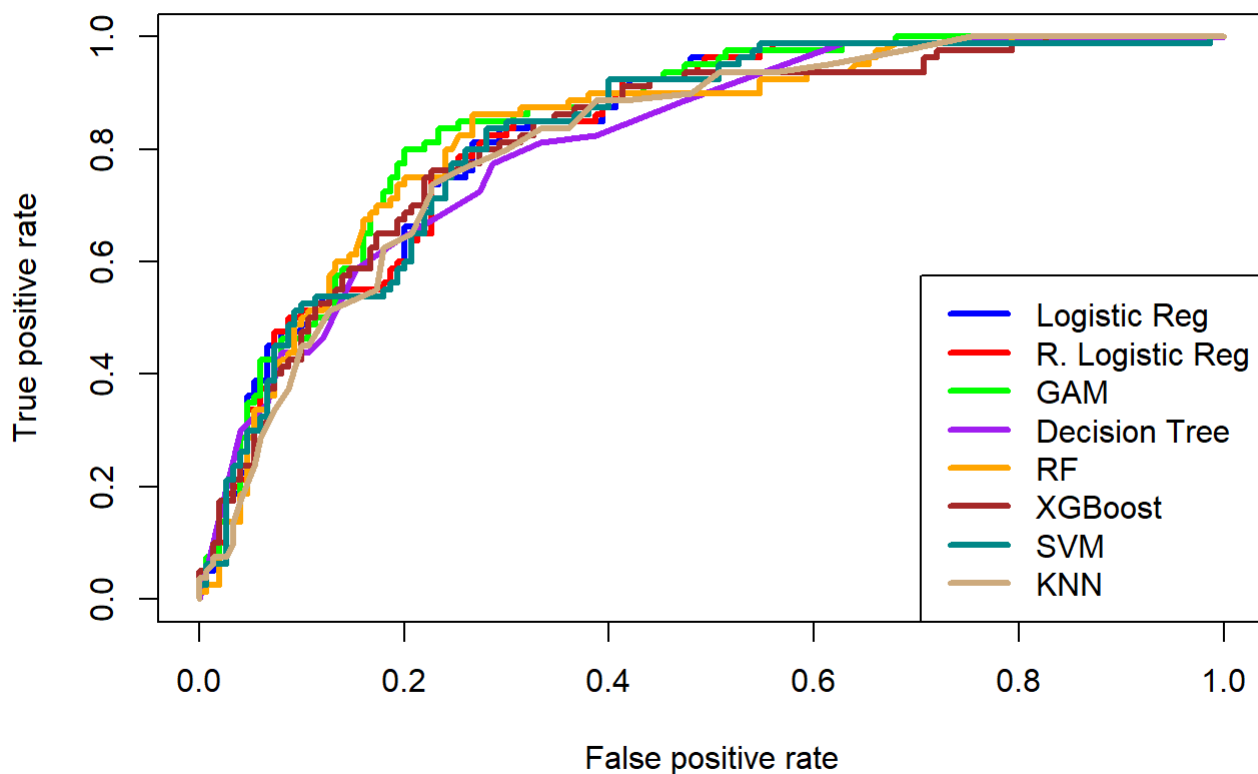
1. Plot ROC curve of each method:

```

plot(0, type="n", main="ROC curve for each method", xlim=c(0, 1), ylim=c(0, 1), xlab="False p
ositive rate",ylab="True positive rate")
lines(fpr_glm_test, tpr_glm_test, col="blue", type="l", lty=1, lwd=3)
lines(fpr_glm_lasso_test, tpr_glm_lasso_test, col="red", type="l", lty=1, lwd=3)
lines(fpr_gam_test, tpr_gam_test, col="green", type="l", lty=1, lwd=3)
lines(fpr_dtree_test, tpr_dtree_test, col="purple", type="l", lty=1, lwd=3)
lines(fpr_rf_test, tpr_rf_test, col="orange", type="l", lty=1, lwd=3)
lines(fpr_xgb_test, tpr_xgb_test, col="brown", type="l", lty=1, lwd=3)
lines(fpr_svm_test, tpr_svm_test, col="darkcyan", type="l", lty=1, lwd=3)
lines(fpr_knn_test, tpr_knn_test, col="burlywood3", type="l", lty=1, lwd=3)
legend("bottomright", legend=c("Logistic Reg", "R. Logistic Reg", "GAM", "Decision Tree", "R
F", "XGBoost", "SVM", "KNN"), col=c("blue", "red", "green", "purple", "orange", "brown", "dar
kcyan", "burlywood3"), lty=1, lwd=3)

```

ROC curve for each method



2. Plot decision curve of each method:


```
df_ <- data.frame(y_glm_test_pred, y_glm_lasso_test_pred, y_gam_test_pred, y_dtree_test_pred,
y_rf_test_pred, y_xgb_test_pred, y_svm_test_pred, y_knn_test_pred, y_test)

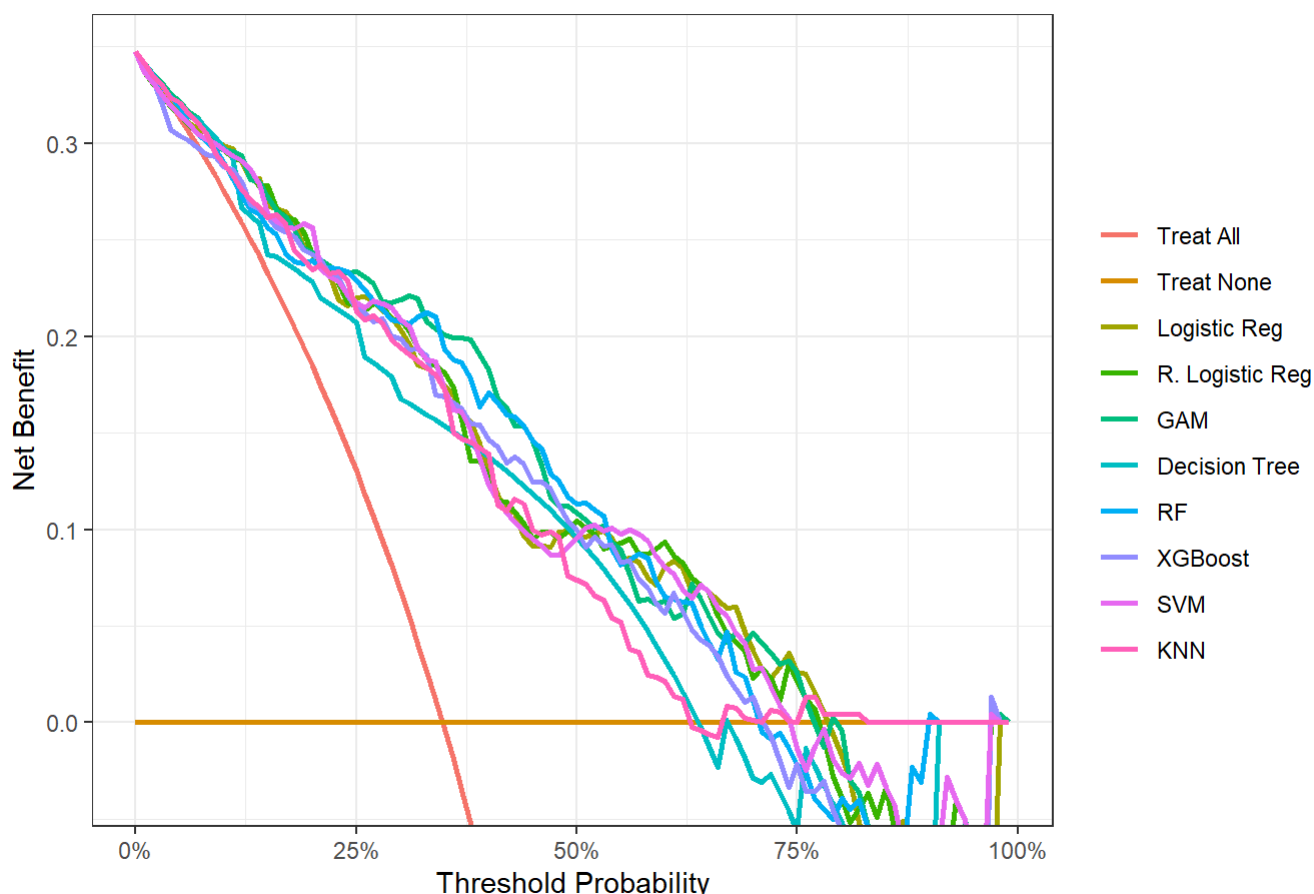
colnames(df_)[2] <- 'y_glm_lasso_test_pred'

# Calculate disease prevalence in the population:
p = sum(test$Outcome == 1)/nrow(test)

dcurves::dca(y_test ~ ., data = df_, label = list(y_glm_test_pred = "Logistic Reg", y_glm_lasso_test_pred = "R. Logistic Reg", y_gam_test_pred = "GAM", y_dtree_test_pred = "Decision Tree", y_rf_test_pred = "RF", y_xgb_test_pred = "XGBoost", y_svm_test_pred = "SVM", y_knn_test_pred = "KNN"), prevalence = p) %>%
  plot(smooth = FALSE) + geom_line(linewidth = 1) + labs(title = "Decision curve for each method") +
  theme(plot.title = element_text(face="bold", hjust = 0.5))
```

```
## Assuming '1' is [Event] and '0' is [non-Event]
```

Decision curve for each method



3. Criteria to determine the decision threshold for the Logistic regression method:

- Youden index criterion:

```
# Optimal threshold obtained by maximizing youden index
j_threshold <- function(y_true, y_pred) {
  roc_obj <- pROC::roc(y_true, y_pred)
  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities
  thresholds <- roc_obj$thresholds
  j <- tpr - fpr
  j <- sort(j, decreasing = TRUE, index.return = TRUE)
  j_ordered <- data.frame(j = j[1], thresholds = thresholds[rank(j$ix)])
  for (k in 1:nrow(j_ordered)){
    if (is.finite(j_ordered$thresholds[k])) {
      return(j_ordered$thresholds[k])
      break
    }
  }
}

cj_glm <- j_threshold(y_test, y_glm_test_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
cj_glm
```

```
## [1] 0.2931594
```

- **Closest to (0,1) criterion:**

```
# Optimal threshold obtained by maximizing sensitivity and specificity to the extent that both are closest to 100%
d_threshold <- function(y_true, y_pred) {
  roc_obj <- pROC::roc(y_true, y_pred)
  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities
  thresholds <- roc_obj$thresholds
  d <- sapply(1:length(fpr), function(i) sqrt((1 - tpr[i])^2 + (fpr[i])^2))
  d <- sort(d, decreasing = FALSE, index.return = TRUE)
  d_ordered <- data.frame(d = d[1], thresholds = thresholds[rank(d$ix)])
  for (k in 1:nrow(d_ordered)){
    if (is.finite(d_ordered$thresholds[k])) {
      return(d_ordered$thresholds[k])
      break
    }
  }
}

cd_glm <- d_threshold(y_test, y_glm_test_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
cd_glm
```

```
## [1] 0.2931594
```

- **Symetric point criterion:**

```
# Optimal threshold obtained by finding the point where sensitivity and specificity are equivalent
s_threshold <- function(y_true, y_pred) {
  roc_obj <- pROC::roc(y_true, y_pred)
  fpr <- 1 - roc_obj$specificities
  tpr <- roc_obj$sensitivities
  thresholds <- roc_obj$thresholds
  s <- abs(fpr + tpr - 1)
  s <- sort(s, decreasing = FALSE, index.return = TRUE)
  s_ordered <- data.frame(s = s[1], thresholds = thresholds[rank(s$ix)])
  for (k in 1:nrow(s_ordered)){
    if (is.finite(s_ordered$thresholds[k])) {
      return(s_ordered$thresholds[k])
      break
    }
  }
}

cs_glm <- s_threshold(y_test, y_glm_test_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
cs_glm
```

```
## [1] 0.3248605
```

4. Plot ROC curve, decision curve and optimal thresholds for the **Logistic regression** method:

```
# Plot ROC curve:
roc_obj <- pROC::roc(y_test, y_glm_test_pred)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```

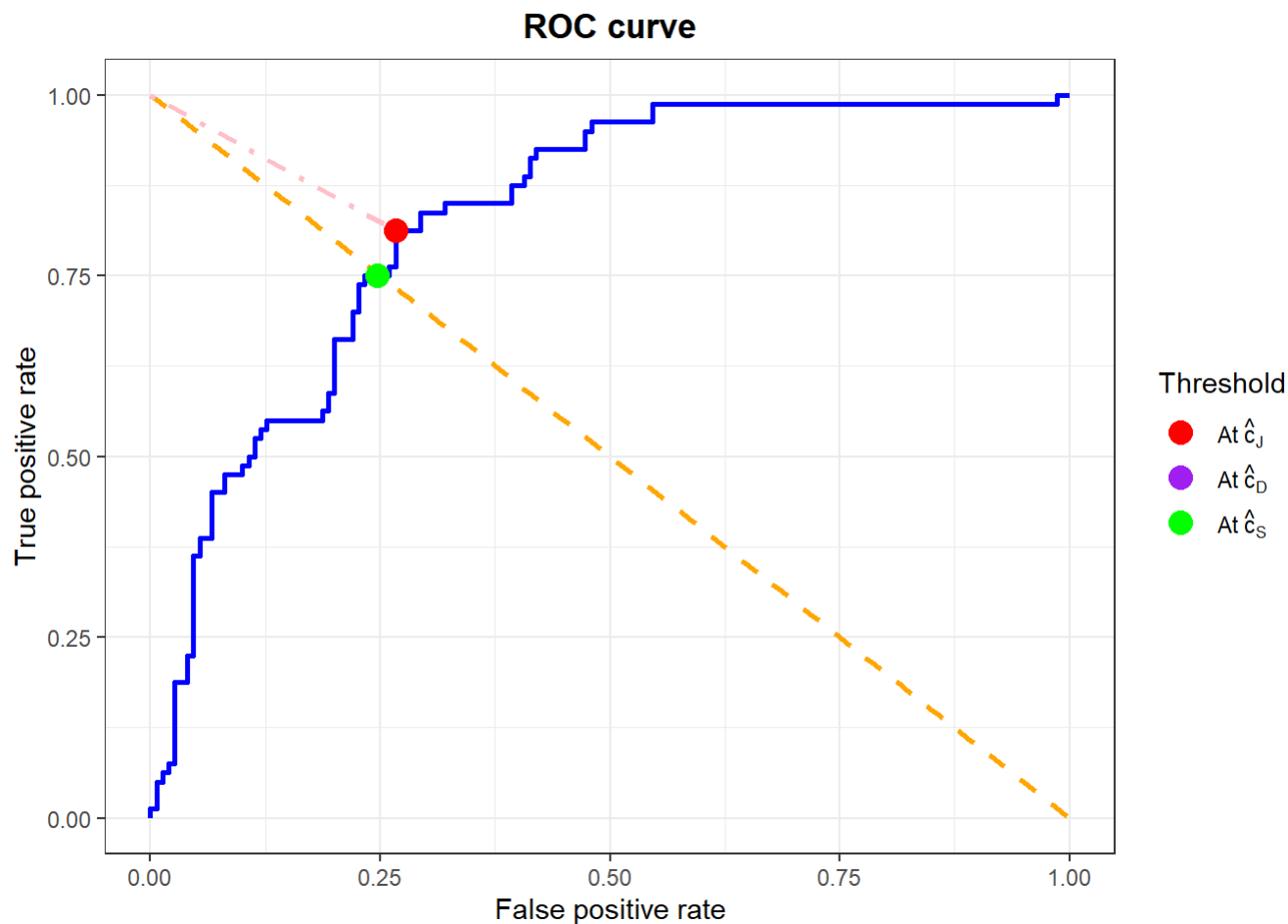
thresholds_glm <- roc_obj$thresholds
roc_data <- data.frame(fpr = rev(1 - roc_obj$specificities), tpr = rev(roc_obj$sensitivitie
s))

for (i in 1:length(roc_obj$thresholds)){
  if (roc_obj$thresholds[i] == cj_glm){
    specificity_j <- roc_obj$specificities[i]
    sensitivity_j <- roc_obj$sensitivities[i]
  }
  if (roc_obj$thresholds[i] == cd_glm){
    specificity_d <- roc_obj$specificities[i]
    sensitivity_d <- roc_obj$sensitivities[i]
  }
  if (roc_obj$thresholds[i] == cs_glm){
    specificity_s <- roc_obj$specificities[i]
    sensitivity_s <- roc_obj$sensitivities[i]
  }
}

diagonal_data <- data.frame(x = seq(0, 1, length.out = 100), y = 1 - seq(0, 1, length.out = 1
00))
thresholds_data <- data.frame(x1 = 0, x2 = 1 - specificity_d, y1 = 1, y2 = sensitivity_d)
sp_se_j <- data.frame(sp = specificity_j, se = sensitivity_j)
sp_se_d <- data.frame(sp = specificity_d, se = sensitivity_d)
sp_se_s <- data.frame(sp = specificity_s, se = sensitivity_s)

ggplot(data = roc_data, aes(x = fpr, y = tpr)) +
  geom_line(linetype = "solid", linewidth = 1, col = "blue") +
  geom_line(data = diagonal_data, aes(x = x, y = y), linetype = "dashed", linewidth = 1, col
= "orange") +
  geom_segment(data = thresholds_data, aes(x = x1, y = y1, xend = x2, yend = y2), linetype =
"dotdash", linewidth = 1, col = "pink") +
  geom_point(data = sp_se_j, aes(x = 1 - sp, y = se, col = "c_j"), size = 4, show.legend = TR
UE) +
  geom_point(data = sp_se_d, aes(x = 1 - sp, y = se, col = "c_d"), size = 4, show.legend = TR
UE) +
  geom_point(data = sp_se_s, aes(x = 1 - sp, y = se, col = "c_s"), size = 4, show.legend = TR
UE) +
  labs(title = "ROC curve", x = "False positive rate", y = "True positive rate") +
  scale_color_manual(name = "Threshold", labels = unname(TeX(c("At  $\hat{c}_{J}$ ", "At  $\hat{c}_{D}$ ", "At  $\hat{c}_{S}$ "))), values=c("red", "purple", "green")) +
  theme_bw() +
  theme(plot.title = element_text(face="bold", hjust = 0.5))

```



```
# Plot decision curve

# Net benefit function
nb <- function(y_true, y_pred, c, prev){
  roc_obj <- pROC::roc(y_true, y_pred)
  for (i in 1:length(roc_obj$thresholds)){
    if (roc_obj$thresholds[i] == c){
      se_c <- roc_obj$sensitivities[i]
      sp_c <- roc_obj$specificities[i]
      break
    }
  }
  phi_c <- se_c * prev - (1 - sp_c) * (1 - prev) * c/(1 - c)
  return(phi_c)
}

df_glm <- data.frame(y_glm_test_pred, y_test)
nb_j <- data.frame(cj = cj_glm, nbj = nb(y_test, y_glm_test_pred, cj_glm, p))
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
nb_d <- data.frame(cd = cd_glm, nbd = nb(y_test, y_glm_test_pred, cd_glm, p))
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

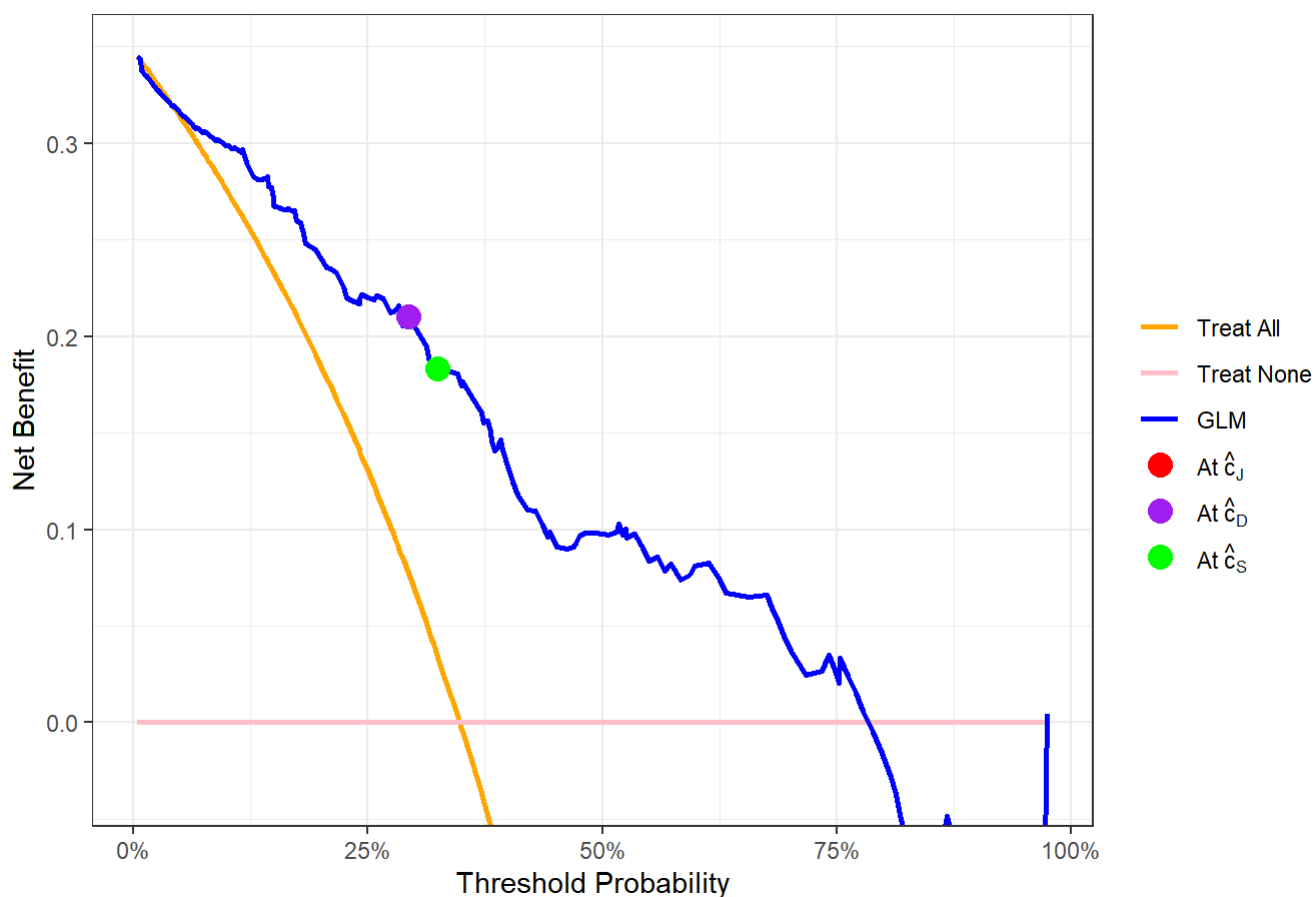
```
nb_s <- data.frame(cs = cs_glm, nbs = nb(y_test, y_glm_test_pred, cs_glm, p))
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
# For the Decision curve plot, use the thresholds values from ROC curve plot to make sure tha
t the optimal thresholds show on the decision curve correctly
dcurves::dca(y_test ~ y_glm_test_pred, data = df_glm, label = list(y_glm_test_pred = "GLM"),
thresholds = thresholds_glm, prevalence = p) %>%
  plot(smooth = FALSE) + geom_line(linewidth = 1) + labs(title = "Decision curve") +
  geom_point(data = nb_j, aes(x = cj, y = nbj, col = "c_j"), size = 4, show.legend = TRUE) +
  geom_point(data = nb_d, aes(x = cd, y = nbd, col = "c_d"), size = 4, show.legend = TRUE) +
  geom_point(data = nb_s, aes(x = cs, y = nbs, col = "c_s"), size = 4, show.legend = TRUE) +
  scale_color_manual(labels = unname(TeX(c("Treat All", "Treat None", "GLM", "At  $\hat{c}_J$ ",
"At  $\hat{c}_D$ ", "At  $\hat{c}_S$ "))), values=c("orange", "pink", "blue", "red",
"purple", "green")) +
  guides(colour = guide_legend(override.aes = list(linetype = c("solid", "solid", "solid", re
p("blank", 3)), shape = c(NA, NA, NA, rep(16, 3))))) +
  theme(plot.title = element_text(face="bold", hjust = 0.5))
```

```
## Assuming '1' is [Event] and '0' is [non-Event]
```

Decision curve



```
# Finding the optimal threshold
nb_j <- nb(y_test, y_glm_test_pred, cj_glm, p)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
nb_d <- nb(y_test, y_glm_test_pred, cd_glm, p)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
nb_s <- nb(y_test, y_glm_test_pred, cs_glm, p)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
nb_values <- c(cj = nb_j, cd = nb_d, cs = nb_s)

best_threshold <- names(which.max(nb_values)) # Name of the threshold with highest net bene
fit
threshold_map <- list(cj = cj_glm, cd = cd_glm, cs = cs_glm)
c_res <- threshold_map[[best_threshold]] # Get the optimal threshold's value
cat("Optimal threshold:", best_threshold, "=", c_res)
```

```
## Optimal threshold: cj = 0.2931594
```

```
## Example: Predicting the class of a new data point with GLM model
new_obs_ <- X_test_unscaled[32:37,] # Some observations from the test set
new_obs <- scale(new_obs_, center = train_mean, scale= train_sd)
new_obs <- as.data.frame(new_obs)
# In the procedure: You need to preprocess (Convert categorical variables into factors, Fill
in NA values, Standardize, ... in the same method and with the same values calculated from th
e preprocessing step of the training set) the new data if needed
actual_classes <- y_test[32:37]
class_preds <- c()

y_glm_pred = predict.glm(glm_model, new_obs, type = 'response')
y_glm_pred = round(y_glm_pred, digits = 7)
class_preds <- ifelse(y_glm_pred >= c_res, "Diabetic", "Not Diabetic")
actual_classes <- ifelse(actual_classes == 1, "Diabetic", "Not Diabetic")

# Show the observations
obs_names <- paste("Observation", 1:6)
obs_names <- data.frame(Observation = unlist(obs_names))
print(cbind(obs_names, new_obs_))
```

```
##      Observation Pregnancies Glucose BloodPressure SkinThickness Insulin  BMI
## 107 Observation 1           1      96           122           0         0 22.4
## 111 Observation 2           3     171           72           33        135 33.3
## 113 Observation 3           1      89           76           34         37 31.2
## 117 Observation 4           5     124           74           0         0 34.0
## 121 Observation 5           0     162           76           56        100 53.2
## 128 Observation 6           1     118           58           36         94 33.3
##      DiabetesPedigreeFunction Age
## 107                        0.207 27
## 111                        0.199 24
## 113                        0.192 23
## 117                        0.220 38
## 121                        0.759 25
## 128                        0.261 23
```

```
# Final result
```

```
res <- data.frame(obs_names, "Predicted probability" = y_glm_pred, "Predicted class" = class_
preds, "Actual class" = actual_classes)
res
```

```
##      Observation Predicted.probability Predicted.class Actual.class
## 107 Observation 1           0.0140517    Not Diabetic Not Diabetic
## 111 Observation 2           0.6249478      Diabetic    Diabetic
## 113 Observation 3           0.0429568    Not Diabetic Not Diabetic
## 117 Observation 4           0.3510944      Diabetic    Diabetic
## 121 Observation 5           0.8914923      Diabetic    Diabetic
## 128 Observation 6           0.1849983    Not Diabetic Not Diabetic
```